# Génération de graphes aléatoires à l'aide de méthodes Monte-Carlo par chaînes de Markov

Félix Lopez, Nicolas Alam

20 juin 2025

## 1 Introduction

Les graphes aléatoires sont des objets mathématiques essentiels pour modéliser des réseaux dans de nombreux domaines. Leur génération permet d'obtenir des comparatifs pour étudier des phénomènes concrets, comme le routage de paquets dans un réseau de machines ou la diffusion de maladies dans un groupe de personnes.

Par exemple, dans le cadre de l'étude d'un protocole de communication entre machines, on peut chercher à évaluer sa robustesse sur différentes structures de réseaux. Générer des graphes aléatoires permet alors de simuler plusieurs organisations possibles des connexions entre les machines, tout en gardant certaines propriétés fixes comme les degrés des sommets. Cela permet de tester si le protocole se comporte toujours correctement ou efficacement sur des réseaux réalistes

Nous cherchons ici à produire des graphes aléatoires de référence, sans biais. Pour cela le processus d'échantillonage doit être uniformément aléatoire parmi l'ensemble des graphes conformes à des contraintes données.

Lorsqu'on souhaite générer un graphe aléatoire, il faut d'abord choisir un modèle qui précise les propriétés structurelles que l'on veut respecter. Tous les graphes considérés ici seront supposés simples et non-orientés, Un modèle simple et très étudié est le modèle d'Erdős–Rényi [1], qui regroupe les graphes dont toutes les paire de sommets ont une même probabilité $p$, indépendamment des autres, d'être reliées par une arête. Malgré le fait que ce choix de contrainte faciliterait la génération, il ne nous permettrait pas de conserver les degrés de notre graphe de départ, une partie essentielle de sa structure. Quand on regarde la distribution de degrés, on remarque que le caractère Poissonien des graphes d'Erdös-Rényi s'éloigne des lois de puissance qui interviennent dans de nombreux graphes réels comme des réseaux sociaux ou des réseaux biologiques.

Pour nos tirages, on utilisera donc le *configuration model* [2, 3], à savoir, les graphes simples dont la séquence des degrés est fixée.

Nous commençons par présenter la méthode classique d'appariement des demi-arêtes, dont les limites justifient le choix d'un algorithme MCMC.

Puis, on explique et énonce les propriétés nécessaires au fonctionnement de cette seconde approche avant de proposer une extension à celle-ci.

Enfin, nous implémentons et analysons ces deux algorithmes de génération, en suivant pour la méthode MCMC, le protocole expérimental proposé par U. Dutta et A. Clauset dans leur article *"Convergence criteria for sampling random graphs with specified degree sequences"* (*Journal of Computational and Graphical Statistics*, 2021) [4].

# 2 Approches théoriques

L'échantillonage d'un graphe du *configuration model* est un problème difficile, il n'existe pas à ce jour, de méthode connue permettant de produire de manière uniformément aléatoire et en temps polynomial, un graphe réalisant une séquence de degrés quelconque. Nous implémentons ici deux approches : la première repose sur l'appariement aléatoire des demi-arêtes (ou *stubs*), méthode classique de construction dans ce modèle, et la seconde est une méthode basée sur les chaînes de Markov Monte-Carlo (MCMC), qui modifie progressivement le graphe obtenu tout en préservant la séquence de degrés.

## 2.1 Méthode constructive du *configuration model*

Dans cette première approche, on part d'un graphe vide $G' = (V', E')$ où $V' = V$ et $E' = \emptyset$ que l'on va petit à petit peupler d'arêtes.

On commence par évaluer la séquence de degrés de notre graphe de départ $G$, puis on crée une liste $L$ où : chaque sommet $v \in V$ apparaît $\deg(v)$ fois, correspondant aux demi-arêtes dont $v$ est l'extrémité. On remarque que $L$ contient $2m = 2 \cdot |E|$ éléments.

Puis on tire à la suite des paires de demi-arêtes $u, v$ que l'on enlève de la liste $L$ avant d'ajouter $\{u, v\}$ à $E$, sauf si $u = v$ ou $\{u, v\}$ y est déjà, auquel cas on annule l'opération, vide le graphe de toute arête, et on recommence le processus de zéro.

Nous détaillons la procédure complète dans l'algorithm 1.

### 2.1.1 Problème des distributions hétérogènes

Ce tirage est uniforme dans le *configuration model* des multi-graphes à boucles or on ne veut générer ici que des graphes simples : sans boucles ni multi-liens. On pourrait naïvement les supprimer après l'appariement mais cela entraînerait plusieurs problèmes :

- La séquence de degrés serait modifiée.
- On perdrait l'uniformité de l'échantillonage : les sommets de plus fort degrés sont plus susceptibles d'apporter des boucles et multi-liens.

Pour pallier à cela, on itère l'exécution en recommençant à zéro si une boucle ou un multi-lien est detecté, jusqu'à obtenir un graphe simple. L'uniformité est bien préservée car l'ensemble des graphes simples est compris dans celui des multi-graphes à boucles.

Malgré tout, on remarque que la probabilité d'échec de l'algorithm 1 est très élevée lorsque la séquence de degré est hétérogènes, ce qui est très fréquent dans l'étude de réseaux réels, rendant cette méthode inutilisable.

Cela justifie le recours à une méthode alternative, décrite dans la section suivante. Les performances de l'exécution répétée de l'algorithm 1 pour générer des graphes simples seront étudiées dans la partie expérimentale du rapport.

## 2.2 Algorithme MCMC

Dans cette seconde approche, nous ne construisons pas un graphe à partir de zéro comme dans la méthode d'appariement classique (algorithm 1), mais nous partons d'un graphe simple réalisant la séquence de degrés imposée, et nous le modifions progressivement. L'objectif est d'approcher un échantillonnage aléatoire aussi uniforme que possible. Pour cela on utilise un algorithme basé sur les méthodes Monte-Carlo par chaîne de Markov, qui procède par répétition de modifications aléatoires sur le graphe jusqu'à atteindre l'uniformité recherchée.

À chaque itération, on tire uniformément, deux arêtes du graphe, puis on les croise, en échangeant deux de leurs extrémités (2-*swap*). Si cet échange crée une boucle ou un multi-lien, on l'annule et on passe à l'itération suivante.

### 2.2.1 Définitions et notations utiles

Avec $G = (V, E)$ le graphe simple de départ.

— **Le $k$-*swap***
On tire uniformément $k$ arêtes toutes différentes $\{\{x_1, y_1\}, \ \dots \ , \{x_k, y_k\}\} \subset E$, ainsi qu'une permutation $\sigma \in S_k$.
Et on remplace dans $E$, les arêtes tirées par $\{\{x_1, y_{\sigma(1)}\}, \ \dots \ , \{x_k, y_{\sigma(k)}\}\}$.
Exemple d'un 2-*swap* dans la figure 1.

— **Le nombre de triangles**
Par triangle on entend : un triplet $\{v_1, v_2, v_3\}$ de sommets distincts, tous reliés entre eux, c'est à dire $\{\{v_1, v_2\}, \{v_2, v_3\}, \{v_1, v_3\}\} \subset E$
Dans un premier temps, on compte le nombre de triangles avec l'algorithm 2, puis on l'ajuste après un 2-*swap* avec l'algorithm 3.

— **L'assortativité**
L'assortativité mesure la corrélation entre le degré d'un sommet, et ceux de ses voisins.
On la calcule dans un premier temps avec l'**??**, puis on l'ajuste après un $k$-*swap* avec l'**??**.

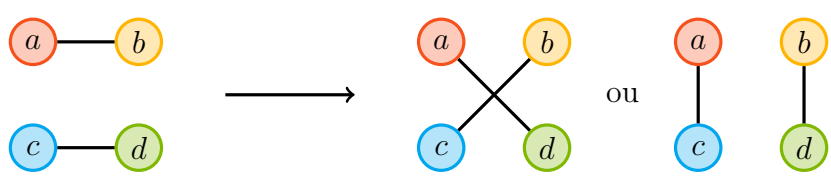


FIGURE 1 – exemple des issues d'un 2-*swap*
On note $G_0$ le graphe de gauche et $E_0 = \{\{a, b\}, \{c, d\}\}$ l'ensemble d'arêtes associées. Soit $G_1$ le graphe résultant d'un 2-*swap* sur $G_0$. $E_1$ peut valoir $\{\{a, d\}, \{b, c\}\}$ ou $\{\{a, c\}, \{b, d\}\}$, les deux seules permutations possibles, à probabilités égales.

### 2.2.2 Propriétés de la chaîne

L'algorithme MCMC s'appuie sur une chaîne de Markov que nous construisons ici sur l'espace des graphes du *configuration model*.

**États :** Chaque état correspond à un graphe $G_t = (V, E_t)$ où $G_0$ est le graphe de départ. Tous sont simples et réalisent la séquence de degrés imposée.

**Transitions :** À chaque itération, on choisit uniformément deux arêtes différentes : $\{\{u_1, v_1\}, \{u_2, v_2\}\} \subset E$ que l'on les remplace par une permutation de leurs extrémités : soit $\{\{u_1, v_2\}, \{u_2, v_1\}\}$ soit $\{\{u_1, u_2\}, \{v_1, v_2\}\}$ à probabilité égale. Si la permutation choisie génère une boucle ou un multi-lien, on l'annule. Il est important de remarquer que cet échange que l'on appelle ici : 2-*swap* (figure 1) , préserve la séquence de degrés.

La convergence de cette chaîne vers une distribution stationnaire unique, indépendante de l'état initial, n'est garantie que si la chaîne est ergodique, c'est à dire si les trois conditions suivantes sont remplies.

**Irréductibilité :** À condition que la séquence de degrés imposée soit réalisable, chaque graphe simple la réalisant doit être accessible depuis un autre par une suite de 2-*swap*. Ce resultat est démontré dans : [citation]

**Aperiodicité :** Dans chaque état $G_t$, il existe une probabilité non nulle de tirer deux arêtes dont le 2-*swap* formerait une boucle ou un multi-lien, c'est à dire de rester dans l'état $G_t = G_{t+1}$,

**Positive-Récurrence :** L'espace concerné ne contient qu'un nombre fini de graphes simples, or la chaîne de Markov est irréductible, elle est donc automatiquement récurrente-positive.

Ces trois propriétés étant vérifiées, la chaîne est ergodique. Nous devons maintenant nous pencher sur une propriété supplémentaire : la symétrie, afin de montrer que cette distribution stationnaire est bien uniforme.

**Symétrie -** Soient $G$ et $G'$ deux graphes simples réalisant la séquence de degrés. On note $P_{G,G'}$ la probabilité de transition de $G$ à $G'$.

- si $G \neq G'$ et si il existe un 2-*swap* les reliant de la sorte : $G \xrightarrow{2\text{-}swap} G'$ alors on remarque que le choix du 2-*swap* est symétrique, tout comme celui de la permutation, car le même 2-*swap* appliqué à $G'$ donne $G$ et est tiré avec la même probabilité. Et donc $P_{G,G'} = P_{G',G}$ .
- si $G \neq G'$ et si il n'existe pas de 2-*swap* les reliant alors $P_{G,G'} = 0 = P_{G',G}$
- si $G = G'$, c'est à dire si le 2-*swap* échoue, alors $P_{G,G'} = P_{G',G}$

**Réversibilité -** La chaîne étudiée est ergodique, définie sur ensemble fini d'états et à transitions symétriques, on en déduit qu'elle est aussi réversible. Sa distribution stationnaire est bien uniforme sur les graphes du *configuration model*.

Ayant établi l'existence d'un état stationnaire vers lequel la chaîne converge. Il est désormais nécessaire de d'implémenter un critère d'arrêt, chargé d'évaluer la convergence et de mettre fin au processus de *swap* une fois l'état stationnaire atteint.

### 2.2.3 Mesure structurelle

La mise en place d'un critère d'arrêt repose sur le choix d'une mesure structurelle, non triviale, représentative de l'évolution de la chaîne. Cette mesure, nous sert à prélever des échantillons à intervalles réguliers sur les graphes de la chaîne. En les analysant via un test statistique, on peut détecter la convergence de la chaîne une fois l'état stationnaire atteint, et à mettre fin au processus de 2-*swap*. Pour des raisons pratiques, on favorise les mesures structurelles aux meilleurs temps de calcul, qui peuvent idéalement être réajustées localement après un 2-*swap*, nous évitant un re-calcul complet.

On décide de suivre dans un premier temps l'évolution du nombre de triangles du graphe (sous-sous-section 2.2.1), initialisé avec l'algorithm 2 et ajusté après un 2-*swap* avec l'algorithm 3.

## 2.3 Contrainte supplémentaire

Après avoir travaillé sur la génération de graphes du *configuration model*, nous nous intéressons à une version plus contraignante du problème, en fixant le nombre de triangles des graphes en plus de leur séquence de degrés.

Conserver un motif, et en particulier toutes ses occurrences permet de préserver des propriétés structurelles du graphe d'origine avec un contrôle beaucoup plus fin qu'en figeant juste la séquence de degrés. On pourrait par exemple dans le cadre d'une analyse de réseaux réels très approfondies, vouloir que l'agencement de certains clusters, reste inchangé dans les modèles de références générés. La solution que l'on présente ici correspond à une version allégée de ce problème car le motif choisi reste très petit.

### 2.3.1 Problèmes posés par la nouvelle contrainte

Comme précédemment, pour garantir l'existence d'un unique état stationnaire uniforme, on doit s'assurer que la chaîne de Markov définie sur ce nouvel espace vérifie bien les quatre propriétés d'irréductibilité, de symétrie, d'apériodicité, et de positive-récurrence. Ces deux dernières se démontrent de la même façon.

Cependant, lorsqu'on impose un nombre de triangles fixé, certains motifs apparaissent dans le graphe, bloquant toute tentative de 2-*swap* et invalidant ainsi l'irréductibilité de la chaîne. Par exemple, si le graphe contient plusieurs composantes connexes dont un triangle isolé, tout 2-*swap* permutant une des arêtes du triangles avec celle d'une autre composante décrémenterait le nombre de triangles de 1, annulant la transition. Ce triangle ne sera donc jamais altéré par un 2-*swap*.
(potentielle figure pour illustrer)

### 2.3.2 Construction d'une nouvelle chaîne

Pour pallier à ce problème, on utilise comme transition un $k$-*swap* légèrement différent, à $k$ variable, proposé par L. Tabourier et J. Karadayidans leur article *"Probabilistic k-swap method for uniform graph generation beyond the configuration model"* (*Journal of Complex Networks*, 2024) [5].
Soit $G_t = (V, E_t)$ le graphe d'état au temps $t$, à chaque itération on tire un $k$ selon une

loi de Zipf [**?**] tel que $2 \leq k \leq m = |E|$ puis on réalise un $k$-*swap*. Comme précédemment, si le nombre de triangles reste inchangé, on conserve la modification, sinon on la rejette.

Grâce aux $k$-*swap*, la nouvelle chaîne est irréductible sur l'espace des graphes à contraintes fixées. Contrairement au cas précédent (sous-sous-section 2.2.2), l'irréductibilité ne repose plus sur l'existence d'une longue suite de transitions. La chaîne munie de $k$-*swap* peut désormais atteindre n'importe quel graphe $G'$ du *configuration model* en une seule transition $G \xrightarrow[k\text{-}swap]{} G'$. (voir démonstration dans [5, Section 3.1]).

La symétrie de cette nouvelle chaîne est aussi conservée.
Il suffit de prendre $G = (V, E)$ un graphe de départ, $2 \leq k \leq |E|$, $\mathcal{E}_k \subset E$ le nombre et l'ensemble d'arêtes à permuter et $\sigma \in S_k$ une permutation. On note $G'$ le graphe résultant du $k$-*swap* sur $G$ que l'on suppose sans échec. Il existe un $k$-$swap^{-1}$ inverse sur $G'$ avec $\sigma^{-1} \in S_k$ et $\mathcal{E}'_k \subset E'$ l'ensemble des arêtes de $\mathcal{E}_k$ après permutation. Comme précédemment, $P_{G,G'} = P(G \xrightarrow[k\text{-}swap]{} G') = P(G' \xrightarrow[k\text{-}swap^{-1}]{} G) = P_{G',G}$

Quant à la mesure structurelle, ne pouvant plus utiliser le nombre de triangles désormais fixe, on choisit d'étudier l'assortativité (sous-sous-section 2.2.1).

# 3 Implémentation et analyse

## 3.1 Méthode constructive

On affirme plus tôt que l'hétérogénéité des degrés d'un graphe est source de boucles et multi-liens lorsque l'on éxecute l'algorithm 1. Pour vérifier cette hypothèse, on met en place un test en traitant deux graphe aux densités très proches :

— $G_1$, aux degrés hétérogènes, de densité $p = 3.975 \times 10^{-3}$.
— $G_2$, un graphe d'Erdös-Renyi, de densité $p = 3.962 \times 10^{-3}$.

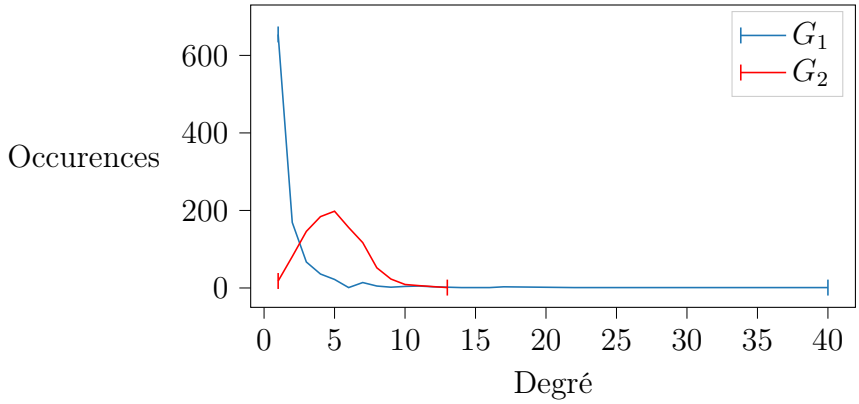


FIGURE 2 – Distribution des degrés de deux graphes : $G_1$ et $G_2$

Ce résultat empirique (figure 3) vient appuyer l'hypothèse : les sommets dont le degré est significativement supérieur à la moyenne ont une forte probabilité de générer des défauts par la méthode d'appariement. La génération sans échec de graphes du *configuration model* aux séquences de degrés très hétérogènes nécessite donc un grand nombre de tentatives.

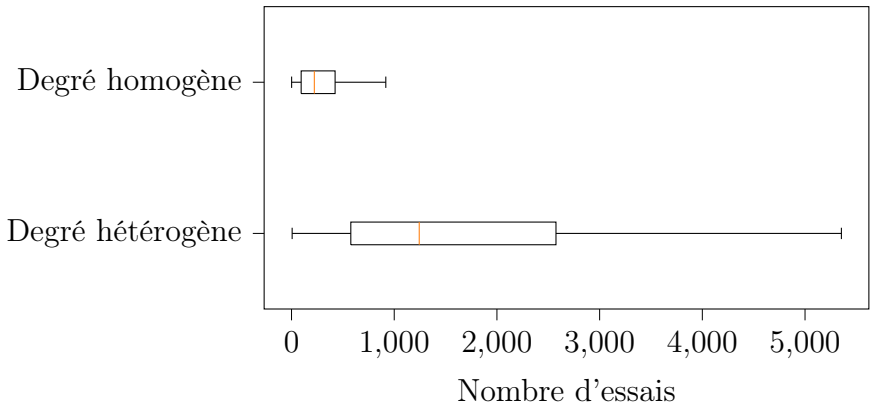


FIGURE 3 – Nombre d'essais de 1000 éxécutions réussies

## 3.2 Méthode MCMC

Dans cette section, nous évaluons empiriquement la convergence de l'algorithme MCMC (sous-section 2.2) dans le but d'obtenir un rang à partir duquel on considère avoir atteint l'état stationnaire de la chaîne, état où la distribution est suffisamment uniforme sur l'ensemble des graphes du *configuration model*. Nous suivons pour cela le protocole expérimental proposé par Dutta et Clauset [4], qui consiste à faire tourner plusieurs chaînes en parallèle, créant un échantillon de graphes : une mesure statistique qui nous sert à évaluer quand on a atteint l'état stationnaire.

On note :

— $G$ le graphe de départ

— $G_{k,t}$ le $t^{\text{ième}}$ état de la $k^{\text{ième}}$ chaîne
Avec $k \leq K$, le nombre de chaînes parallèles.

— $m_{k,t}$ une mesure structurelle prélevée sur le graphe de l'état $G_{k,t}$

### 3.2.1 Protocole

Le principe est le suivant :

- On lance $K$ chaînes de Markov initialisées à $\forall k \leq K,\ G_{k,0} = G$.
- On observe l'évolution d'une mesure structurelle prélevée à intervalles réguliers sur l'ensemble des $K$ chaînes.
- On interromp ou non cette évolution, en fonction du résultat du critère d'arrêt.

### 3.2.2 Choix du *sampling gap*

Chaque chaîne de Markov parcourt un grand nombre d'états dont seule une fraction est retenue pour l'analyse la convergence. Le *sampling gap* ou "écart d'échantillonnage", noté $\eta_0$, désigne le nombre d'itérations entre deux mesures successives, et est dans notre implémentation, commun aux $N$ chaînes parallèles. L'écart $\eta_0$ que le *sampling gap* représente doit être assez grand pour que la corrélation entre deux états qu'il sépare $G_t$ et $G_{t+\eta_0}$ soit négligeable : les mesures faites sur une chaîne avec un *sampling gap* trop petit, pourraient être redondantes et ne déclencher l'arrêt du programme que très tard.

De même, un *sampling gap* trop grand gâcherait du temps de calcul, en poursuivant aussi le processus sur une chaîne ayant déjà atteint l'état stationnaire.

Ce paramètre doit donc être choisi avec précaution, pour cela, on utilise l'arbre de décision [4, Figure 6] qui a été construit expérimentalement pour générer des graphes à séquence de degré fixée à l'aide d'une procédure de recherche automatique.

$\cdots$ valeur retenue pour le graphe utilisé $\cdots$

### 3.2.3 Détection de la convergence

Pour détecter empiriquement la convergence de la chaîne de Markov vers une distribution stable, nous utilisons un test similaire à celui proposé par Dutta et Clauset [4].

Celui-il consiste d'abord à rassembler les mesures prélevés sur les $K$ échantillons, à chaque pas $T\eta_0$, dans des listes $L_T = \{ m_{1,T\eta_0} , m_{2,T\eta_0} , \ldots , m_{K,T\eta_0} \}$ de taille $K$.

Puis à appliquer le test de Kolmogorov–Smirnov non-paramétrique à deux échantillons [6] sur les listes $L_{T-1}$ et $L_T$ afin de déterminer si elles peuvent raisonnablement être considérées comme issues de la même distribution.

Si le test de Kolmogorov–Smirnov rejette l'hypothèse nulle selon laquelle les deux échantillons $L_{T-1}$ et $L_T$ proviennent de la même distribution, on réitère le processus de *swap* sur toutes les chaînes parallèles jusqu'au temps $(T+1)\eta_0$, avant d'appliquer le test de Kolmogorov–Smirnov aux deux échantillons $L_T$ et $L_{T+1}$.

On répète cette opération jusqu'à ce que le test ne puisse plus rejeter l'hypothèse nulle, ce que l'on interprète comme un signe de convergence des chaînes vers leurs distributions stationnaires, à savoir la loi uniforme sur l'ensemble des graphes réalisant la séquence de degrés fixée. Sur n'importe quelle chaîne parallèle, chaque pas $\eta_0$ supplémentaire à partir de ce point est alors considéré comme un tirage indépendant et identiquement distribué depuis le *configuration model*.

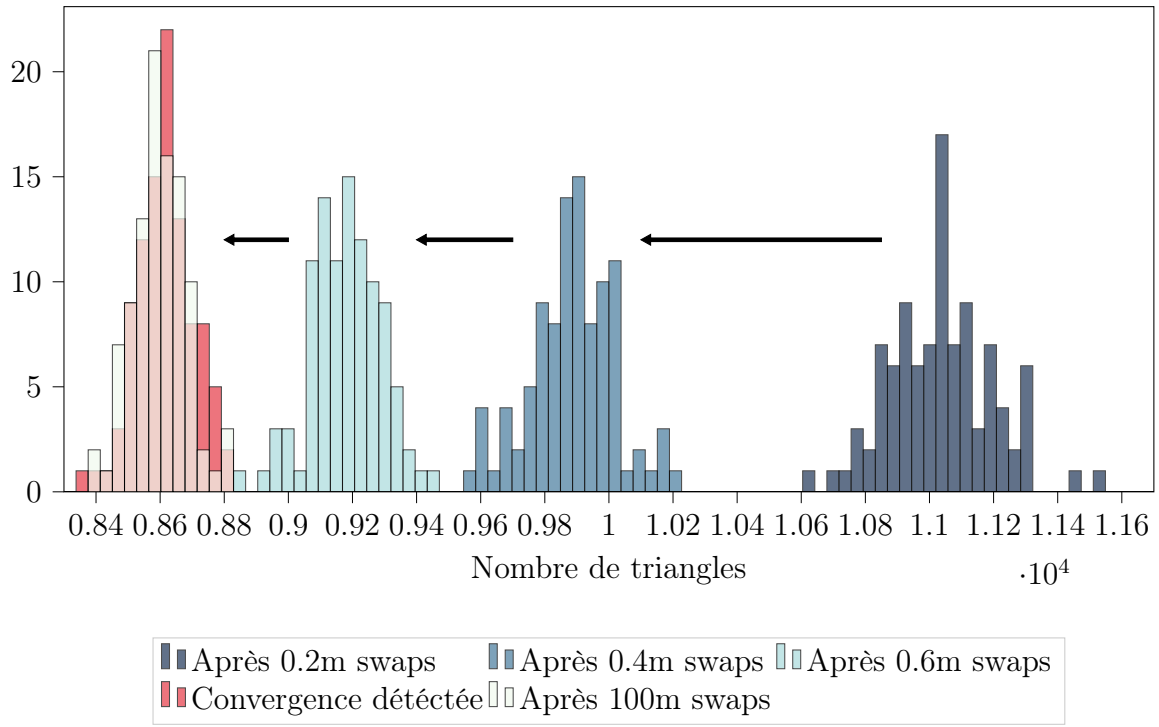


FIGURE 4 – Premier plot de la convergence (à peaufiner)

# Références


[1] P. Erdős and A. Rényi. On random graphs i. *Publicationes Mathematicae Debrecen*, 6 :290–297, 1959.

[2] E. A. Bender and E. R. Canfield. The asymptotic number of labeled graphs with given degree sequences. *Journal of Combinatorial Theory, Series A*, 24(3) :296–307, 1978.

[3] Béla Bollobás. A probabilistic proof of an asymptotic formula for the number of labelled regular graphs. *European Journal of Combinatorics*, 1(4) :311–316, 1980.

[4] Upasana Dutta and Aaron Clauset. Convergence criteria for sampling random graphs with specified degree sequences. *CoRR*, abs/2105.12120, 2021.

[5] Lionel Tabourier and Julien Karadayi. Probabilistic k-swap method for uniform graph generation beyond the configuration model. *Journal of Complex Networks*, 12(1), February 2024.

[6] Frank J. Massey Jr. and. The kolmogorov-smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253) :68–78, 1951.

# A Algorithmes

## A.1 méthode classique de construction du *configuration model* (sous-section 2.1)

**Algorithm 1:** Tentative de formation aléatoire de paires de demi-arêtes sans boucle ni multi-lien

```
input  : H : tableau de 2n demi-arêtes (entiers)
output: H trié en n paires sans boucle ni multi-lien, ou NULL en cas d'échec
begin
    for i ← 0 to n − 1 do
        j_1 ← entier aléatoire dans ⟦0, 2n − 2i − 1⟧;
        j_2 ← entier aléatoire dans ⟦0, 2n − 2i − 2⟧;
        pos_1 ← 2n − 2i − 1 ;    // emplacement pour la première demi-arête
        pos_2 ← 2n − 2i − 2 ;    // emplacement pour la seconde
        échanger H[j_1] avec H[pos_1];
        échanger H[j_2] avec H[pos_2];
        u ← H[pos_2], v ← H[pos_1] ;    // nouvelle paire (u, v)
        if u = v then
            return NULL ;    // boucle détectée
        end
        // vérifie si (u, v) ou (v, u) a déjà été créé
        for k ← 0 to i − 1 do
            a ← H[2k], b ← H[2k + 1];
            if (a = u ∧ b = v) ou (a = v ∧ b = u) then
                return NULL ;    // multi-lien détecté
            end
        end
    end
    return H ;    // tableau de paires valides
end
```

## A.2 Comptage de triangles

**Algorithm 2:** Comptage des triangles

**Entrée:** Un graphe non orienté $G = (V, E)$ représenté par des listes d'adjacence
**Sortie :** Le nombre total de triangles dans $G$

```
nb_triangles ← 0;
for a ← 0 to n − 1 do
    foreach b voisin de a où b > a do
        foreach c voisin de b où c > b do
            if c est voisin de a then
                nb_triangles ← nb_triangles + 1;
            end
        end
    end
end
```

**Algorithm 3:** Mise à jour incrémentale du nombre de triangles après un 2-*swap*

**Entrée:** Graphe $G$ (non orienté, liste d'adjacence)
Deux arêtes à échanger : $(a, b)$ et $(c, d)$
**Sortie :** Mise à jour de $G.triangles$

```
T ← G.triangles;
foreach x ∈ {a, b} do
    foreach y voisin de x où y ≠ b et y ≠ d do
        if (y, b) existe then
            T ← T − 1;
        end
        if (y, d) existe then
            T ← T + 1;
        end
    end
end
foreach x ∈ {c, d} do
    foreach y voisin de x où y ≠ a et y ≠ c do
        if (y, a) existe then
            T ← T − 1;
        end
        if (y, c) existe then
            T ← T + 1;
        end
    end
end
G.triangles ← T;
```